Type Through the Bible (C++ Edition)

A Typing Game of Biblical Proportions

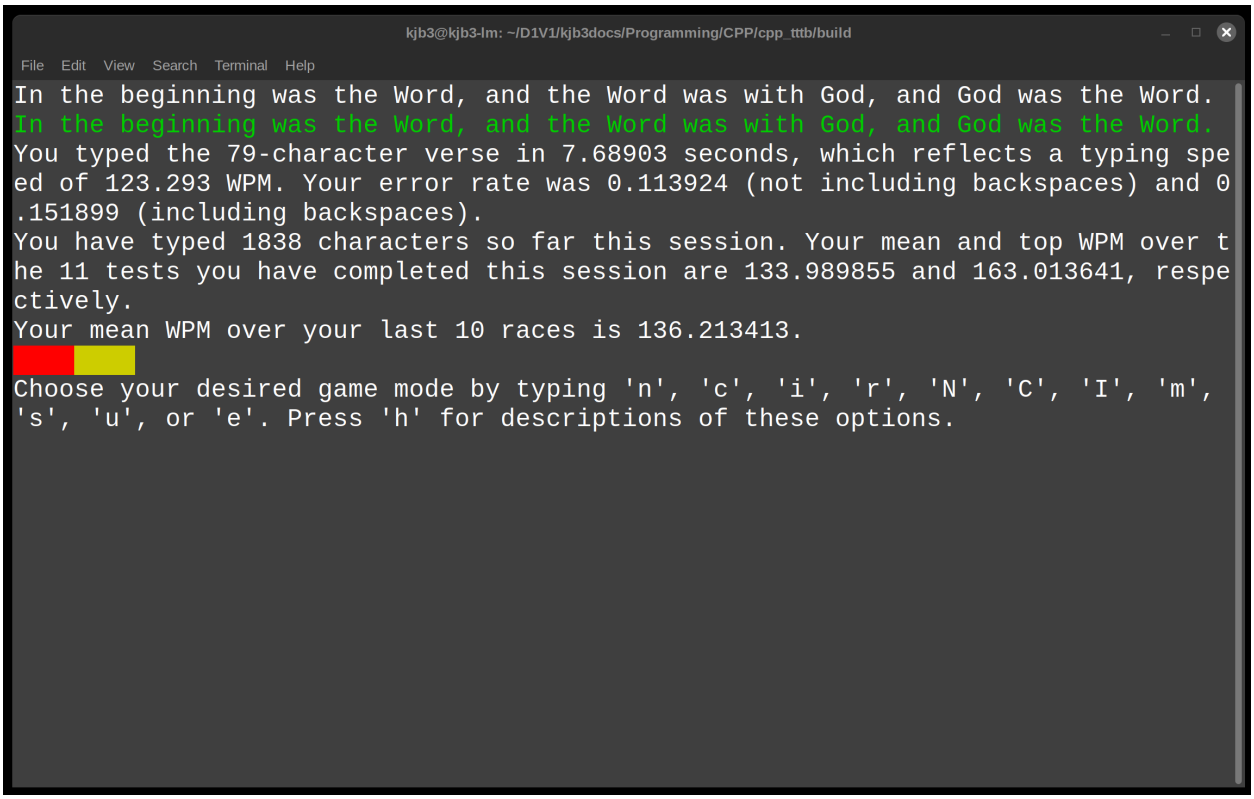
By Ken Burchfiel

Released under the MIT License

Link to GitHub repository

Link to game download page on Itch.io

Link to gameplay demo video



Latest version: 2.0.1

(Note: the in-game files will state that the C++ and complementary Python files are each version 1.03; however, those numbers refer to an earlier versioning system.)

Introduction

Type Through the Bible (TTTB) allows you to practice your keyboarding skills by typing the Bible verse by verse. It contains both single-player and multiplayer modes; in addition, it offers (via a complementary Python script) a wide variety of interactive visualizations.

Download instructions

Linux, Windows, and Apple-silicon OSX binaries of TTTB are available at this page on itch.io . The game is free to download, but donations (while not expected) are greatly appreciated.

Make sure to download both the zipped folder for your operating system *and* the corresponding README (e.g. the file you're reading right now). Once you've unzipped the folder on your local computer, you can

move it to a location of your choice–or simply keep it within the downloads folder. Nothing needs to be installed in order for you to begin playing the game.

(If multiple people will be playing TTTB's single-player mode on your computer, I recommend creating a separate copy of the game for each of them. This will make it much easier for each player to keep track of his or her progress.)

Because TTTB is released under the MIT license, you are free to modify its underlying C++ and Python code, then share your revisions with others (even under a proprietary license). In this case, you'll want to download and compile the source code rather than use these underlying binaries. See 'Compilation instructions' below for more details.

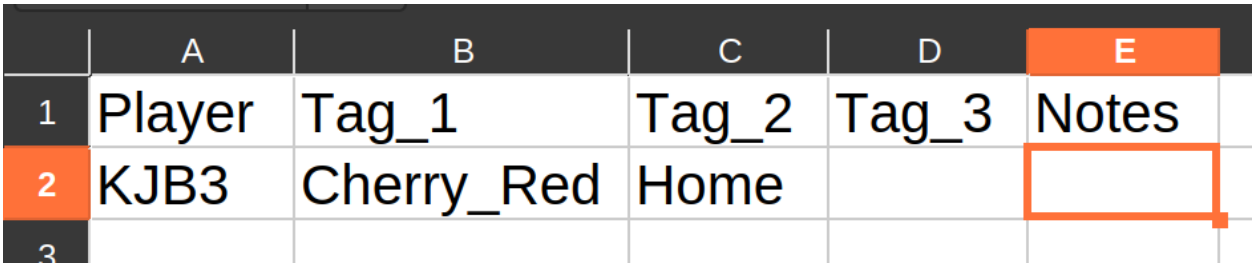
(However, the Bible verses found within this repository are all available within the public domain. See the LICENSE file for more details.)

Mac/OSX users: Make sure to carefully review 'Starting the game' section below for important details on successfully launching TTTB.

Updating your default configuration settings

Once you've downloaded the game, you may want to update your default configuration settings before you begin. That way, certain default values (like your name) will get stored within every test; you'll also have the option to update these settings within single-player games. You can add these default options within the second row (e.g. the row right below the headers) of the game_config.csv file.

Here's what my default settings look like: (I use Tag_1 to store the keyboard I'm using for a test and Tag_2 to store my location. You can choose not to enter any default tags, but I'd recommend at least storing your name within the 'Player option.')



	A	B	C	D	E
1	Player	Tag_1	Tag_2	Tag_3	Notes
2	KJB3	Cherry_Red	Home		
3					

TTTB's folder directory

Here is how TTTB's folders and files are organized:

cpp_tttb/ [the root folder for the game; your exact name may differ]

—-build/

———_internal/ [contains Pyinstaller-related files; these are crucial to include in order for the corresponding Python executable to work correctly, but you shouldn't need to modify them. This folder isn't part of the OSX release, as its contents are compressed within the tttb_py_complement executable.]

———tttb [tttb.exe on Windows; the main TTTB executable that you'll launch in order to start the game]

———tttb_py_complement [tttb_py_complement on Windows; the Python executable that TTTB uses to (1) create single-player and multiplayer visualizations and (2) combine various multiplayer files into single files. This executable can be called either within TTTB or as a standalone executable.]

—-Files/

———MP_Test_Result_Files_To_Combine/

———MP_Word_Result_Files_To_Combine/

———Multiplayer/ [multiplayer results will be stored here.]

———CPDB_for_TTTB.csv

———game_config.csv [This file stores default configuration options, such as your name and various gameplay tags.]

———test_results.csv

———word_results.csv

—-Visualizations/

———Multiplayer/ [Multiplayer visualizations will get stored here.]

———Single_Player/ [Single-player visualizations will get stored here.]

Gameplay instructions

Starting the game

To launch TTTB, you'll first want to open up your command prompt or terminal, then navigate to the program's **build** folder (where the game's 'tttb'/'tttb.exe' executable file is located). On my Linux computer, I can accomplish this via the following steps:

1. Press Ctrl + Alt + T to launch my terminal
2. Enter `cd '/home/kjb3/D1V1/Documents/!Dell64docs/Programming/CPP/cpp_tttb/build'` to navigate to the build folder. (This folder path will of course look different on your end–but it should end in build/, not cpp_tttb or whatever your root TTTB folder is called.)

The Mac steps look very similar; you'll just need to search for your Terminal app if it's not already present within your taskbar.

On Windows, I would instead press the Windows button, then enter `cmd` to bring up the command prompt. I would then enter `cd` followed by the path to my build folder (perhaps encased in double quotes).

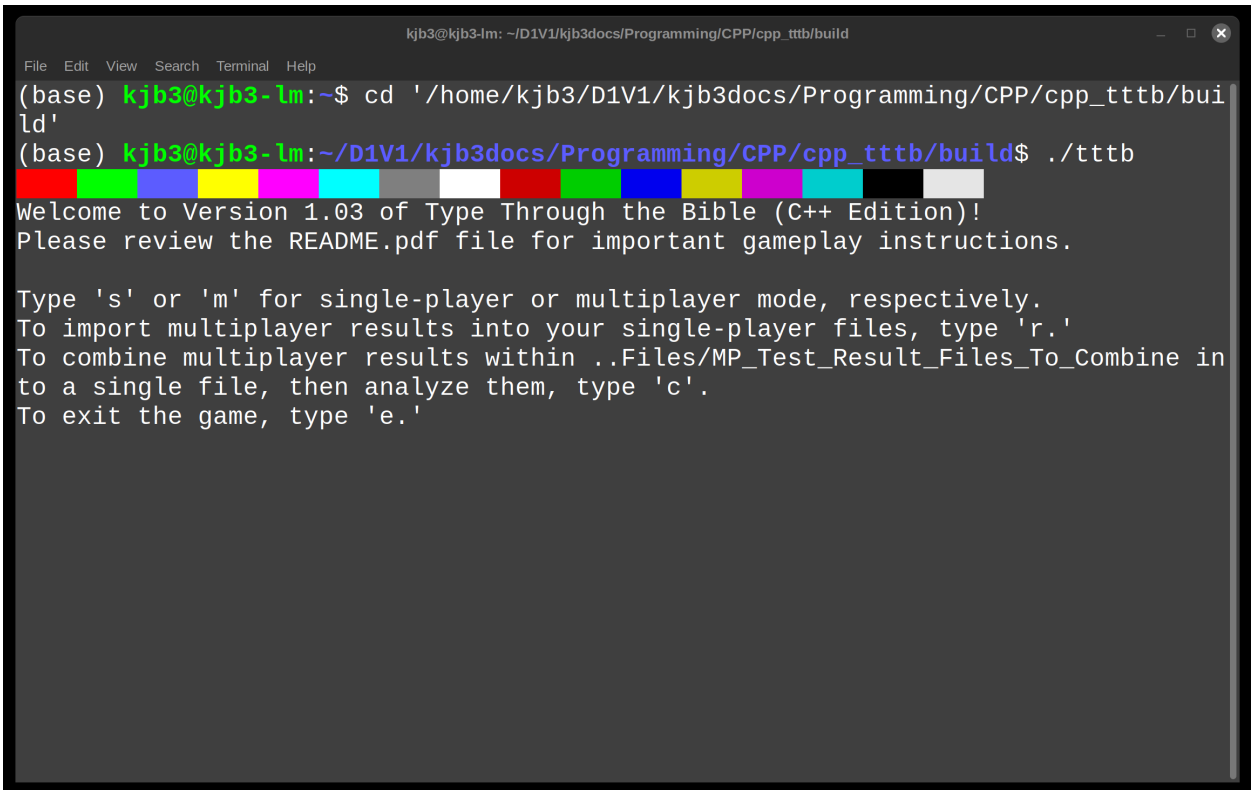
Once you've navigated to this folder, you can launch TTTB by entering the following command (on Linux or OSX):

```
./tttb
```

On Windows, you would instead enter:

```
tttb.exe
```

Once you've completed these steps, you should see the following welcome screen:



However, things are a bit trickier on OSX. When you launch the game *for the first time*, you'll see a box pop up with the message "tttb Not Opened." Here's how you can resolve this issue:

a. Click on the question mark on the top right of this box to open another window with the title "Apple can't check app . . .".
b. Select 'Open Privacy & Security settings for me' within this window.
c. Close the "Apple can't check app" window; hit 'Done' on the original "tttb Not Opened" box; and then, within the Privacy & Security Settings window; select the 'Allow Anyway' option next to the " 'tttb' was blocked" message.
d. Within your terminal, re-enter the `./tttb` command. (You should be able to get back to it by pressing the up arrow.) You'll now see an "Open 'tttb'?" window come up.
e. Click "Open anyway", then enter your account password.
f. You're almost done! You'll likely see a "zsh: segmentation fault" message appear in your terminal. If this occurs, simply enter './tttb' in your terminal one more time, and you should finally be able to enter the game.

Thankfully, during subsequent gameplay sessions, you should *not* need to repeat this process. *However*, at the end of your first session, you'll need to repeat the same steps for the tttb_py_complement executable that gets called to run Python-based data visualization scripts. (After you've completed these steps once, you shouldn't need to perform them again.)

An alternative to all these steps is to download the GitHub clone of TTTB and compile both the C++ and Python code yourself. (See Compilation instructions section below for more details.)

[Note: Windows may also prompt you for your approval before allowing TTTB, or the tttb_py_complement.exe executable, to run. You should only need to do this a few times, but if you keep getting prompted for different Python libraries, let me know by creating a new issue. I can then replace the existing Windows Pyinstaller setup with a single-file setup that I use for the OSX release.]

Single-player gameplay

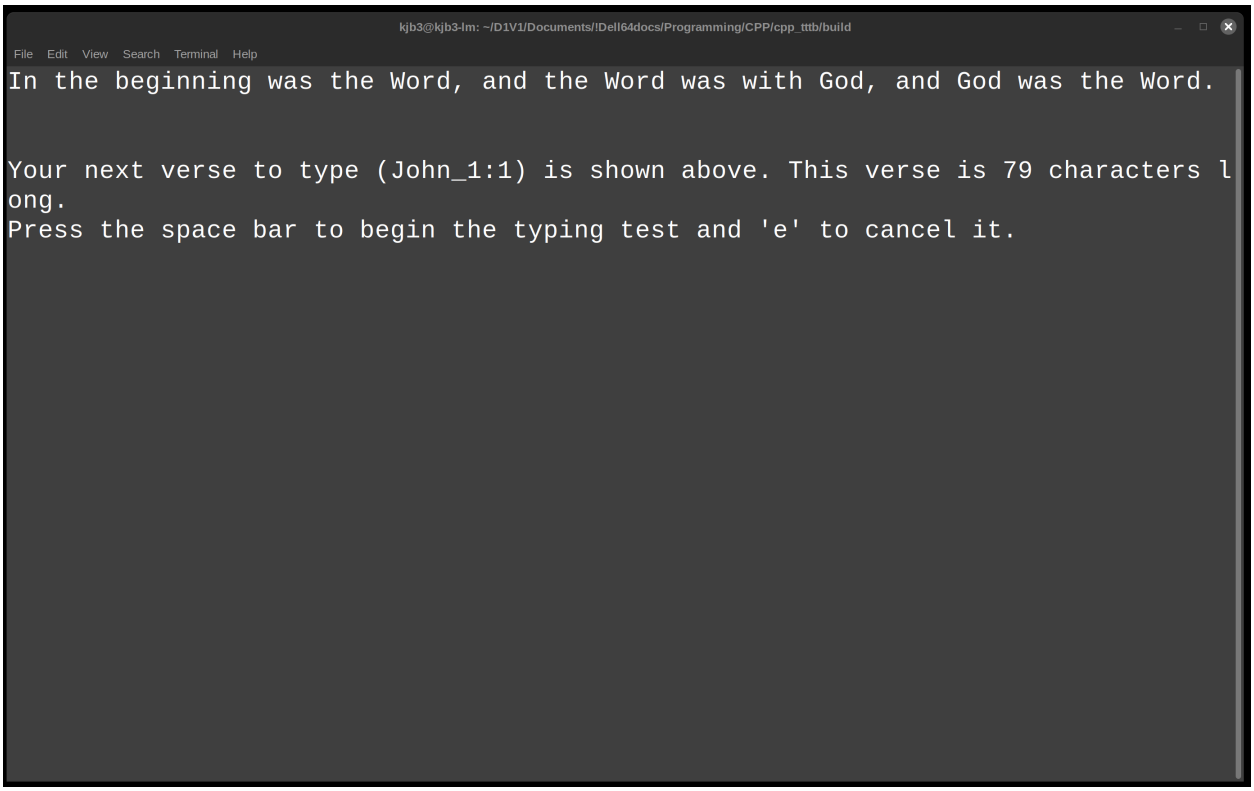
To launch single-player games, press 's' within the welcome screen. You'll then be presented with a number of different single-player modes from which to choose:

- **n**: Type the next untyped verse, then return to the gameplay menu. (This, along with 'N', is a good starting option.)
- **c**: Type the next verse, then return to the gameplay menu. (The main difference between 'c' and 'n' is that the former won't skip over verses that you've already typed at least once.)
- **i**: Type a specific verse ID, then return to the gameplay menu.
- **r**: Repeat the verse you just typed, then return to the gameplay menu.
- **N**: Automatically get directed to the next untyped verse. (This option eliminates the need to press 'n' after each test if you wish to stay in this mode.)
- **C**: Automatically get directed to the next verse.
- **I**: Select a verse from which to begin, then automatically get directed to the verses that follow it.
- **m**: Enter 'untyped marathon mode,' in which you will immediately start a test for the next untyped verse after finishing the current test. (This option, along with 'S', can get pretty exhausting–so consider trying it out only for short periods. Also note that, in the stats that appear under your entry within all marathon modes, 'E&B' stands for 'error and backspace rate'. See 'Understanding error rates' section below for more information.)
- **s**: Enter 'sequential marathon mode,' in which you will immediately start (1) the verse following the one you just typed or (2) (when starting this mode) a verse of your choice.
- **U**: Update game configuration settings.
- **C**: Exit this session and save your progress.

Note: You can exit out of any test, even in marathon mode, by pressing Ctrl+C (not to be confused with Command+C). ***Do not*** simply exit out of the terminal, as your progress then won't be saved.

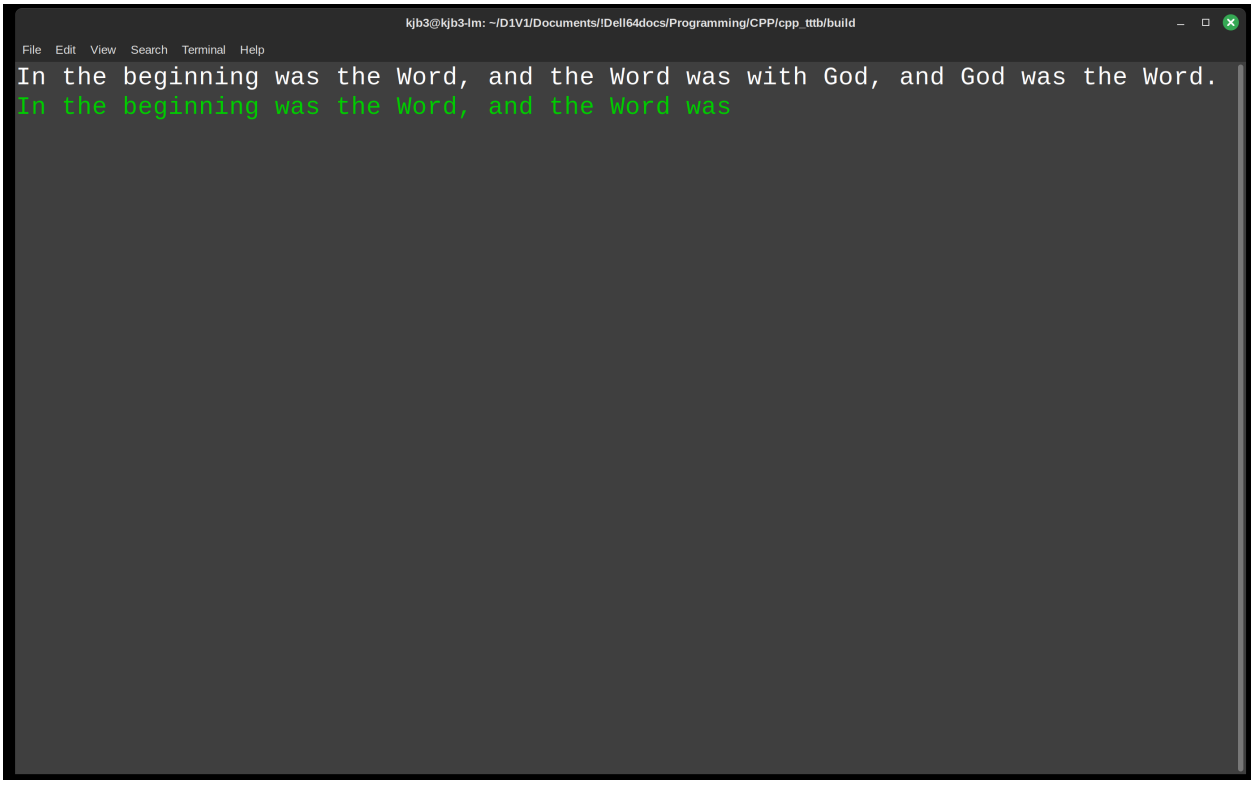
Running typing tests

Within each game mode, you'll be presented with one or more tests. Each of these tests will consist of a single Bible verse that you'll need to type. Unless you're in one of the two 'marathon' modes, you'll have the chance to review the verse before you begin typing it:

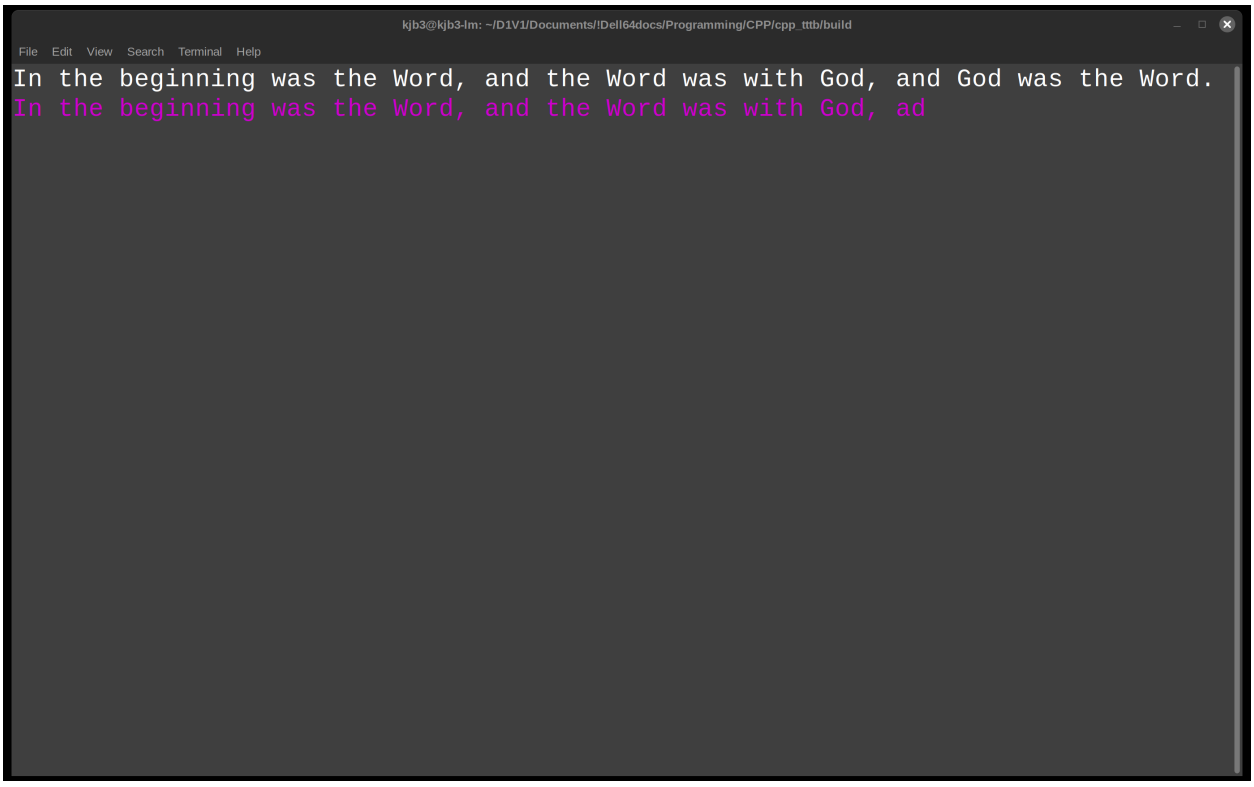


Once you press the space bar, the test's timing clock will start and you'll be able to begin typing. This timing clock will end automatically once you have typed the verse correctly. (In other words, you won't need to press 'Enter' after the test is over.)

You'll get color-coded feedback as you progress through the test. As long as you've typed the verse correctly, your response will be highlighted in green:



However, if you make a typo (even as minor as an extra space), your response will turn magenta in color:



Once you correct the typo (e.g. by hitting backspace), the text will turn green again.

Important: To clear out an entire mistyped word, press Alt + Backspace on Linux and Windows or Fn

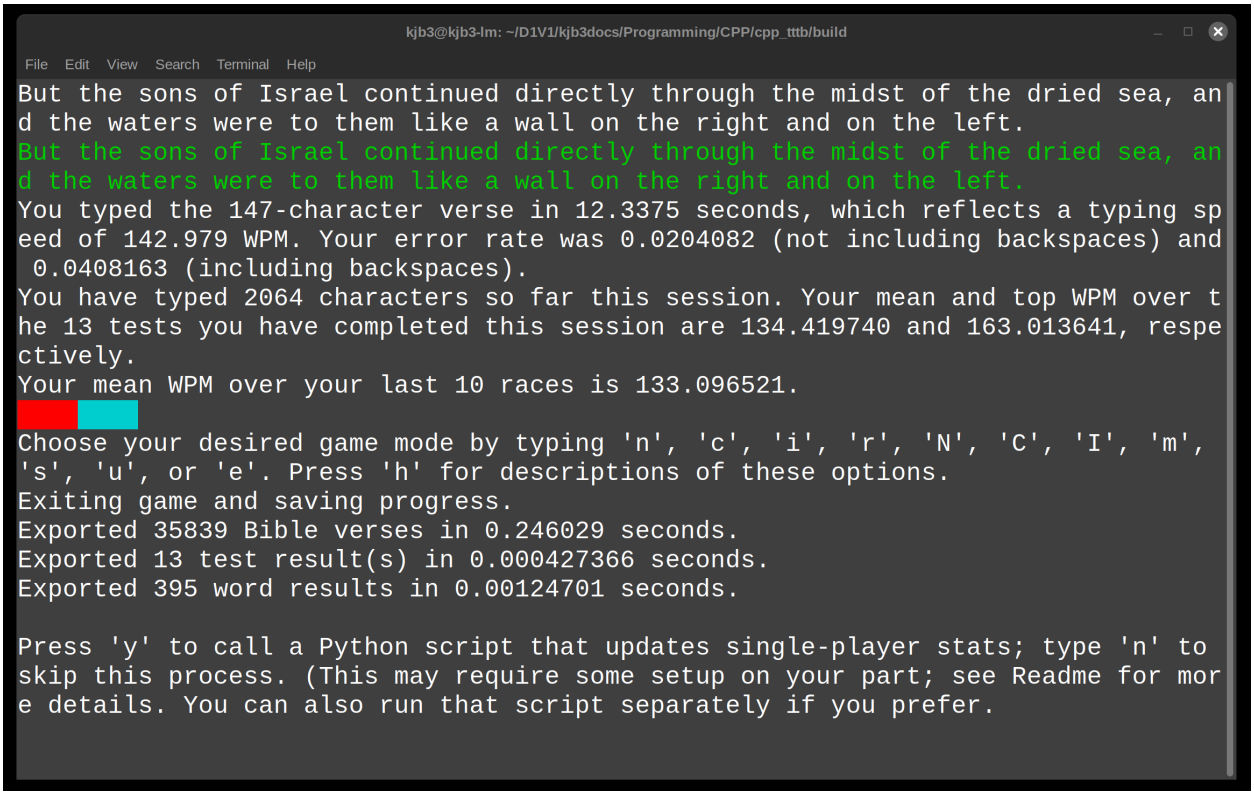
+ Delete on OSX. (I would have liked to make Ctrl + Backspace an option for Linux and Windows, and Command + Delete an option for OSX, but I had trouble implementing these options using the cpp-terminal library; it's possible that they might be added in in a future update.)

After you finish the verse, you'll get feedback on how long it took you type the text; your WPM (defined as ((characters in the verse) / (time, in seconds, you needed to complete the test)) * 60 / 5; and your error rates. (See the very first image within this Readme for an example of what this will look like–though the exact output will vary based on how many tests you have completed.)

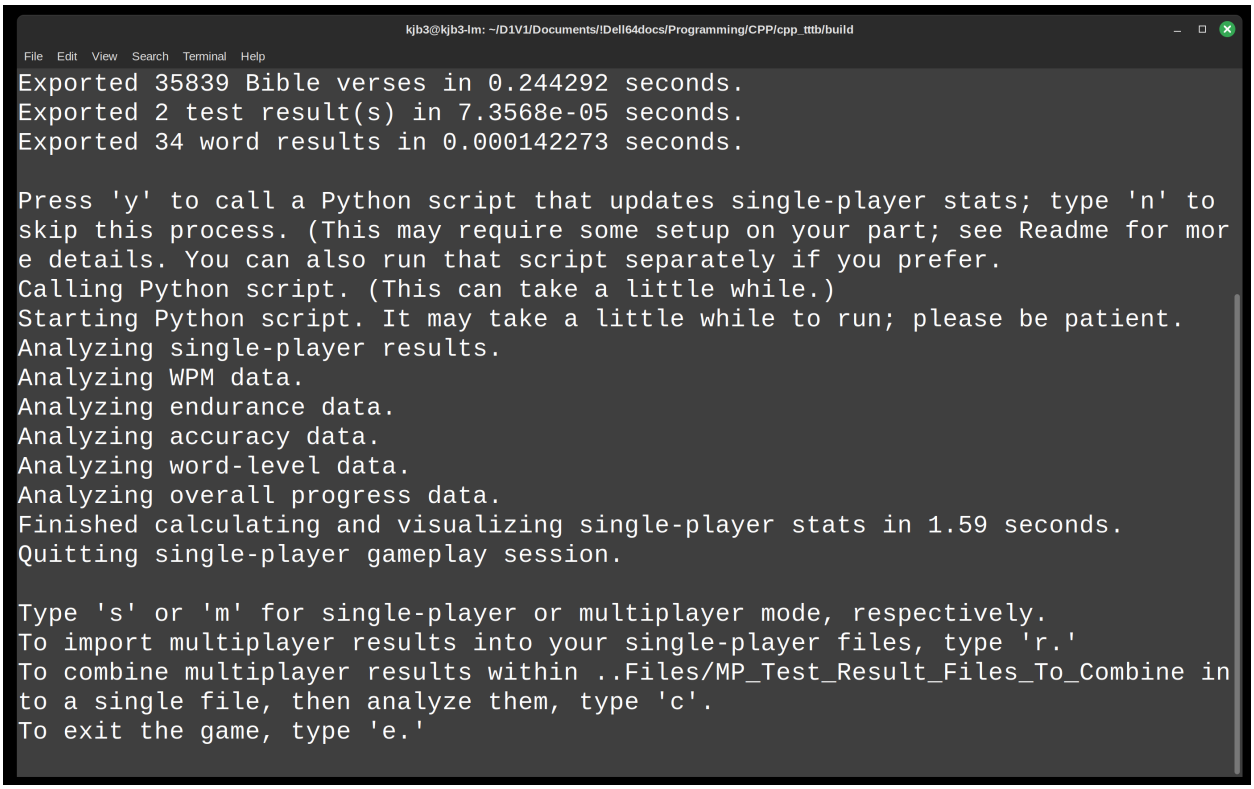
Understanding error rates

TTTB keeps track of two error rates for each verse: one that counts backspaces (which, while not necessarily errors, will still affect your final typing speed), and one that doesn't. The denominator for each rate is the number of characters in the verse. Thus, an error rate of 0.05 means that you had 5 erroneous keypresses for every 100 characters.

Once you're ready to quit out of the game, press 'e'. Your results will then be saved to .csv files.



You'll then have the opportunity to call a Python script that will create interactive visualizations of these results. This shouldn't take terribly long, but if you're in a rush and/or have completed massive numbers of tests, you may not want to call it right now. (You can always call it later; since the script retrieves its data from the .csv files in the Files/ folder, which *always* get updated following a successful exit from single-player mode, declining to visualize your results at this time won't cause any data to get lost.)



Autosave files Every 10 races, TTTB will automatically save test-level and word-level results, along with an updated copy of your CPDB_for_TTTB.csv file, to your Files/ folder. If the game happens to crash, *and* you have completed at least 10 races, you *may* want to:

1. Manually add the data in autosaved_test_results.csv and autosaved_word_results.csv to your main word_results.csv and test_results.csv files. (This will involve opening up both sets of files, then copying the rows from the autosave file into the bottom of the main files.)
2. Replace CPDB_for_TTTB.csv with autosaved_CPDB_for_TTTB.csv.

If, on the other hand, you haven't completed at least 10 races, you *won't* want to copy in these autosave files, as they will reflect data from previous sessions.

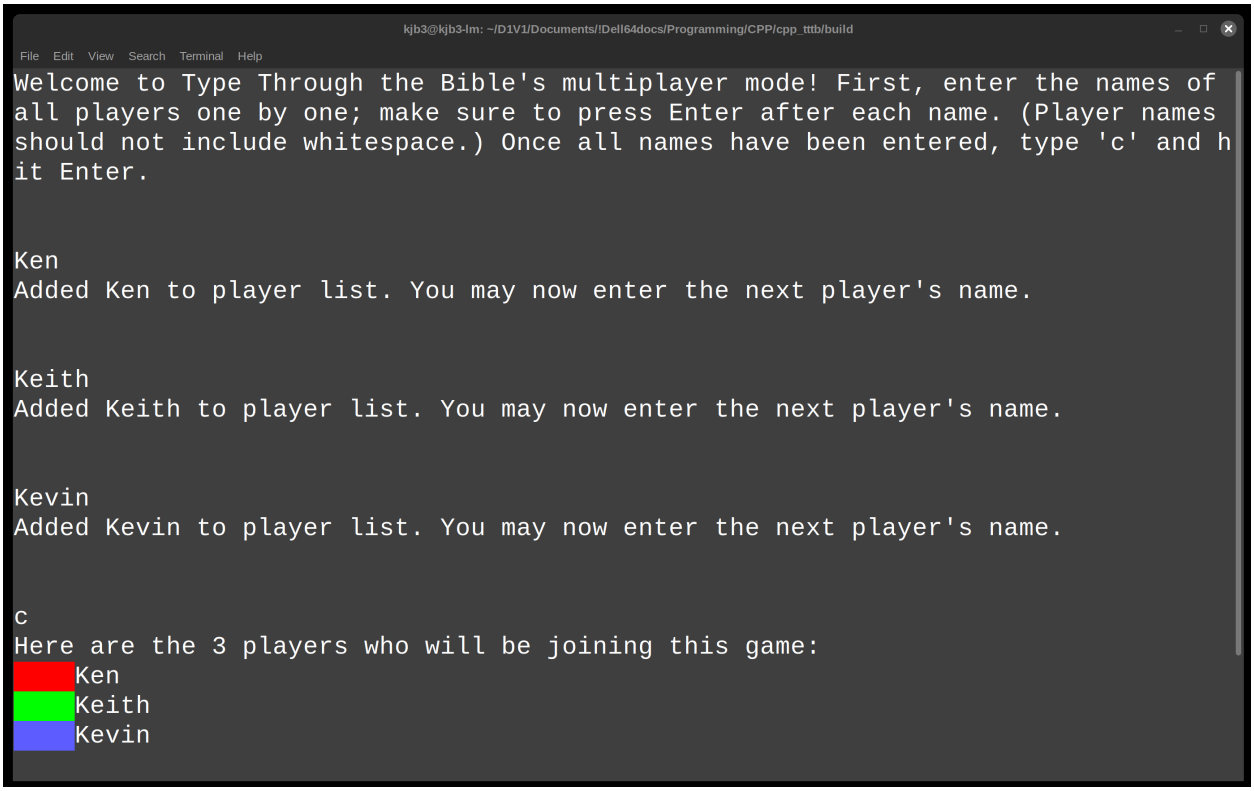
Multiplayer gameplay

Type Through the Bible also supports multiplayer gameplay. There are two methods of performing multiplayer gameplay: single-computer and multi-computer.

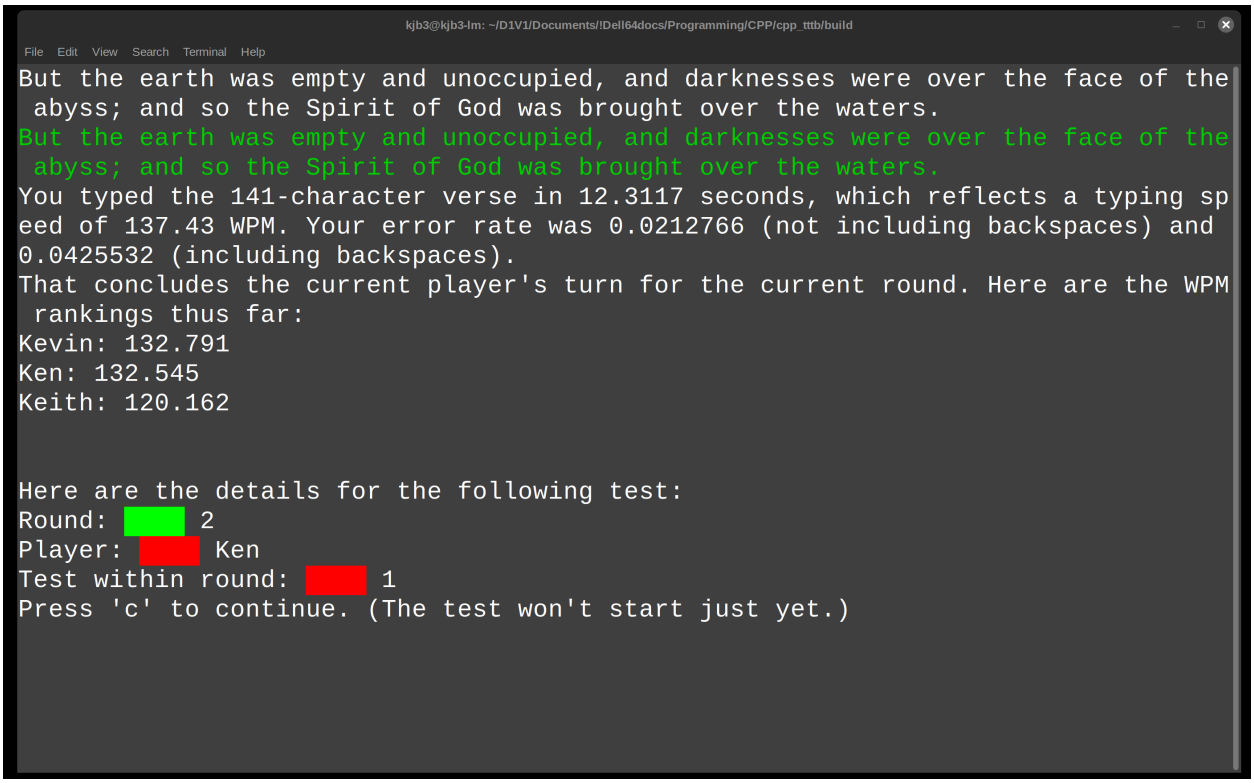
Single-computer multiplayer gameplay

In this method, players will take turns typing on the same computer. This makes for a simpler setup, but is less efficient (at least for larger numbers of players) than the multi-computer setup that I'll discuss shortly.

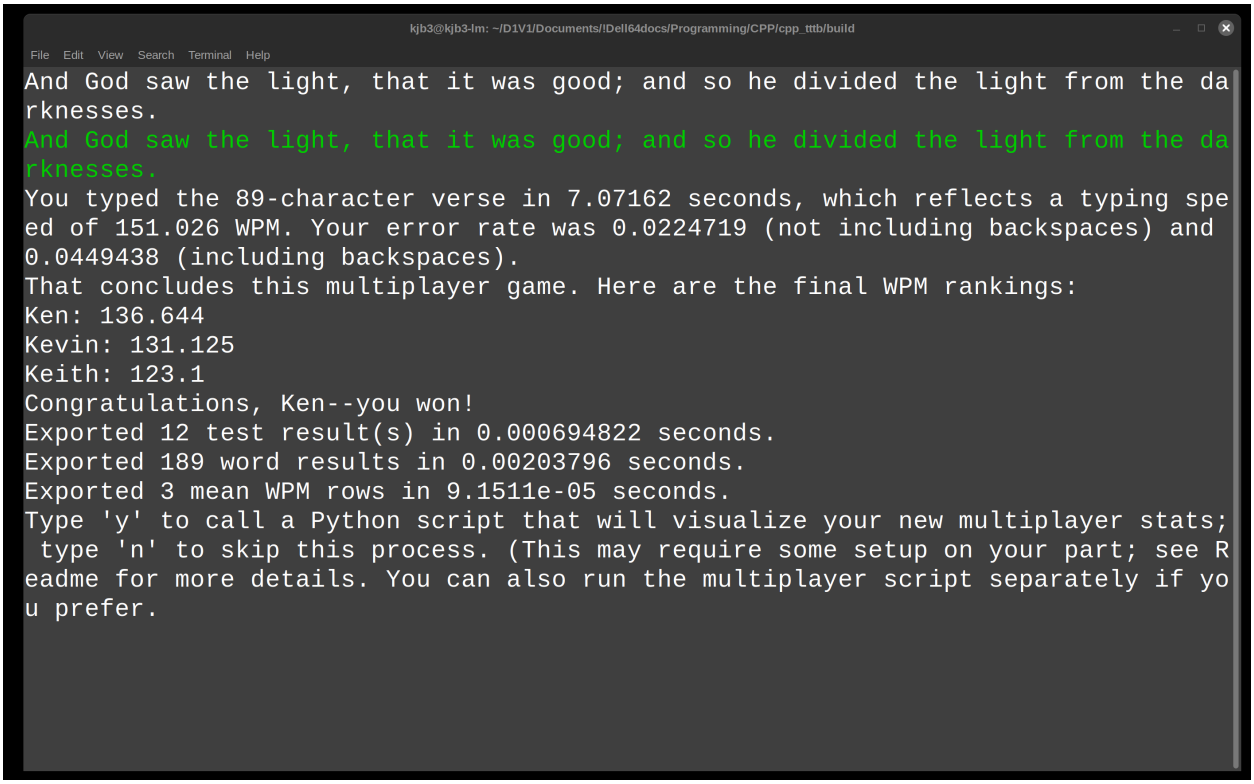
To begin this mode, press 'm' within TTTB's start menu to launch a multiplayer game. You'll be prompted to specify (1) the names of each player; (2) the number of rounds, and tests within rounds, you'd like to play; and (3) the verse ID at which you'd like to begin the game. (There's also an option to type randomly-selected verses. Regardless of whether you wish to type consecutive or randomly-chosen verses, each player will type the exact same sequence of passages.)



Players will then take turns typing the series of Bible verses that you specified during setup. Text notifications will inform you who is typing; make sure to pay careful attention to them so that you don't accidentally type someone else's race (which I did once!).



Once the game ends, you'll get to see everyone's mean WPM; the player with the highest average WPM is the winner. The game will also (1) save a series of multiplayer results files to Files/Multiplayer and (2) ask whether you wish to create visualizations of these files.



Multi-computer multiplayer gameplay

If you have lots of people who wish to play a multiplayer game, then it may be more efficient to use a multi-computer approach. That way, lots of people can play the game at the same time, and they can all compete on the keyboard that they're most comfortable with. (However, given that each player will need to download and install TTTB on their computer beforehand, and that you'll need to perform some additional steps to compile everyone's results, this method may not necessarily save time.)

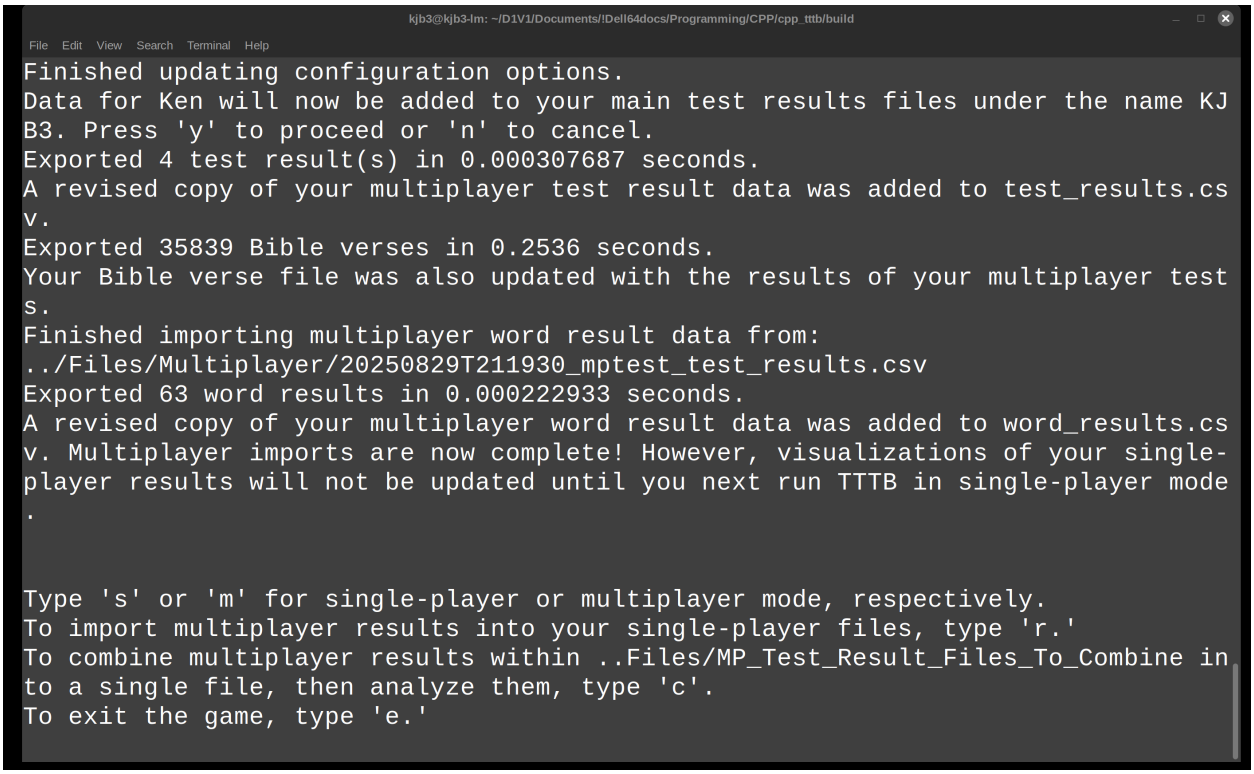
Here's how I recommend going about this method:

1. Have every player launch a *single-player game* within Type The Bible.
2. Decide the verse ID at which you would like to begin the test, then have each player select this ID within the 'I' single-player mode. (This mode, not to be confused with 'i' mode, begins at a specified verse, then automatically presents the tests that follow this verse–whether or not the player has typed them already. This helps ensure that each player will be typing the exact same set of tests–which will be crucial for your subsequent analyses to be valid.)
3. You'll also want to decide how many verses total to type. I recommend choosing a number that's a multiple of 10, since this will make the process of sharing results a bit easier. (You'll learn why shortly.)
4. Once players have finished typing their verses, they should all add their results *for only the races they just typed* to the same folder. This could be an online storage folder; a flash drive; or even a network-attached storage setup. (If your total test count is a multiple of 10, players can simply share their most recent autosave folder, which should contain only the races they just typed. Otherwise, they can simply cut and paste their results into a separate .csv file. Note that there's no need to include headers.)
5. A designated player should then copy all of these test results into his/her 'Files/MP_Test_Result_Files_To_Combine'

folder. Next, he/she should launch TTTB, then select 'c' within the game's start menu and follow the subsequent prompts. TTTB will then run some Python code that (1) converts the copies of multiplayer results into a single file and (2) runs the same visualizations script that it executes for single-computer multiplayer games.

Converting multiplayer data to single-player data

You can also use the 'r' command within the start menu to import multiplayer results into your single-player results if you so choose. (This will be particularly helpful if you completed a long stretch of races within a multiplayer game; importing them into your single-player file will then allow them to count towards your overall progress in typing the entire Bible.)



Analyzing your results

Type Through the Bible offers two main ways to review and evaluate your progress. The first are a series of .csv files that keep track of your results; the second are a set of interactive visualizations that make these results easier to interpret.

CSV files

Your single-player data is stored in the following .csv files within your game's Files folder:

1. 'test_results.csv' (This file provides detailed data on all of your test results, including WPM and accuracy data; start and end times; and verse-related information.)
2. 'CPDB_for_TTTB.csv' (A local copy of the Catholic Public Domain Bible. This file shows the full text of each verse, typed or untyped, as well as (1) the number of times you've typed it and (2) your highest-ever WPM for each verse.
3. 'word_results.csv' (This file provides WPM and accuracy data for each individual word that you typed within each test.)

Meanwhile, your multiplayer result data for each session can be found within Files/Multiplayer. These files will be prefaced with a timestamp and (if provided) a custom string in order to make it easier to link each file to each individual multiplayer session.

Test- and word-level results files are made available for each session, along with a simple '. . . pivot.csv') file that stores each player's mean WPM.

NOTE: Be very careful when opening these files in Excel! I've found that Excel tends to automatically reformat dates within spreadsheets, then save those reformatted dates. These reformatted versions will most likely raise errors when you attempt to use them again. (As an alternative, I recommend LibreOffice Calc, which doesn't appear to attempt to reformat dates. You could also try opening the .csv files within a text editor (since, in the end, they're just text files) in order to further prevent any reformatting nightmares.

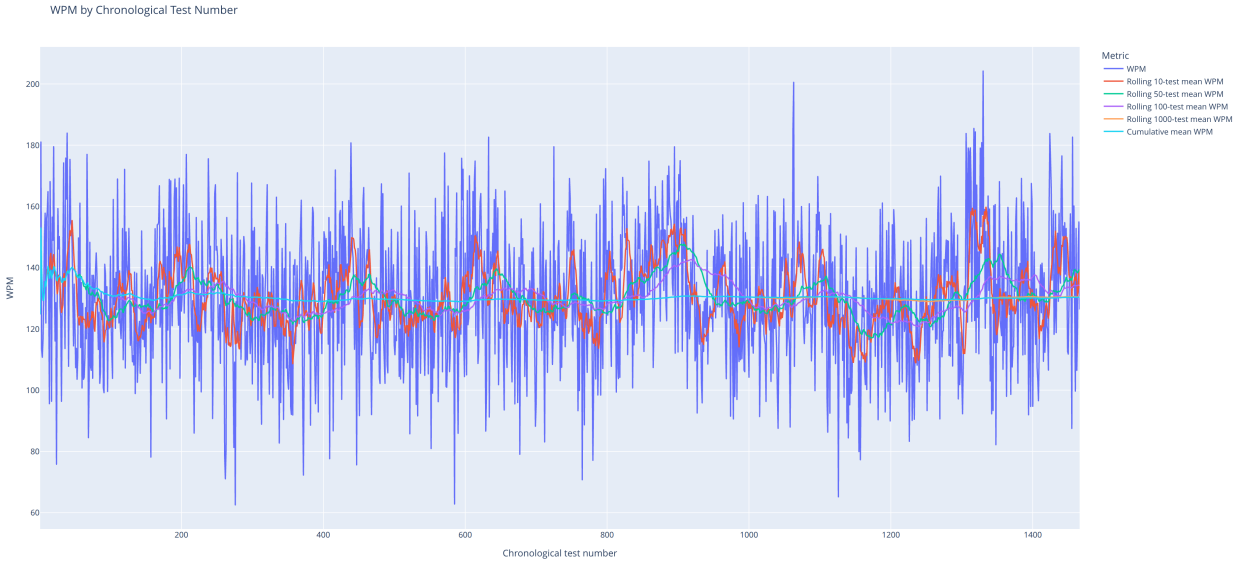
HTML-based interactive charts

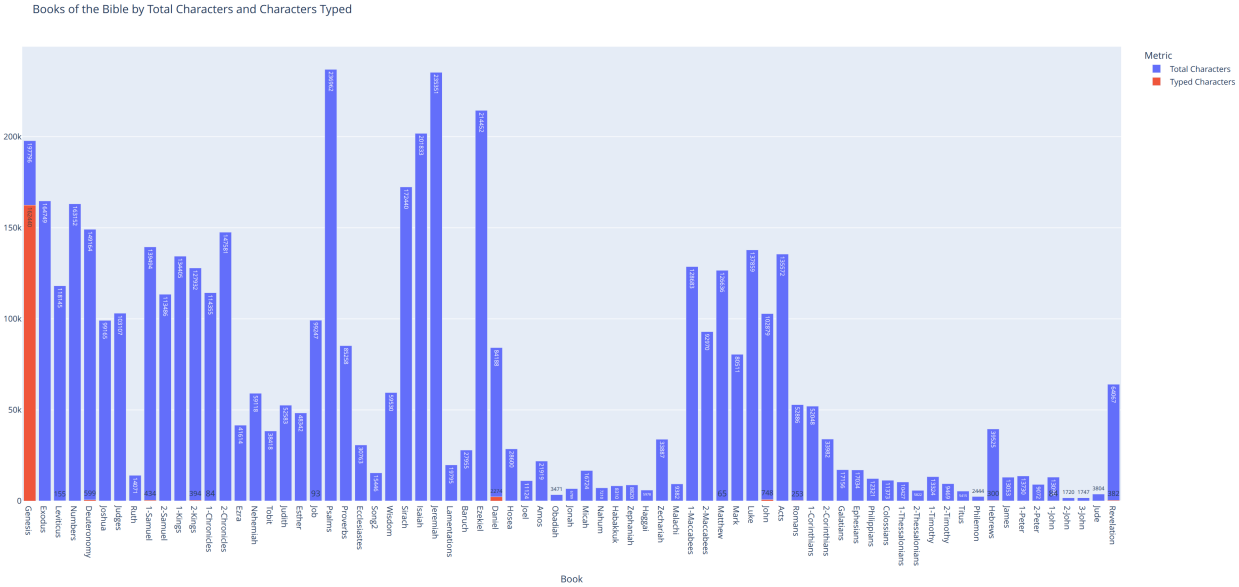
TTTB also provides many interactive, HTML-based visualizations of your results. These are created via a separate Python script that TTTB can call at the end of each single-player and multiplayer game. You can find these charts within the 'Multiplayer' and 'Single_Player' subfolders of the 'Visualizations' folder.

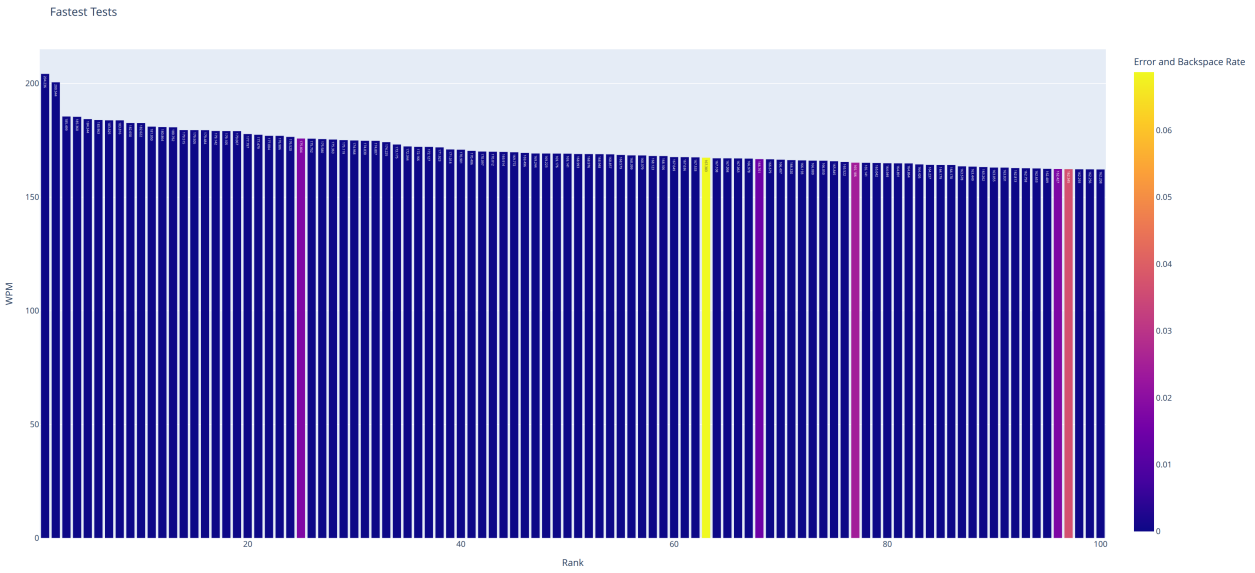
Because these visualizations are HTML-based, you can hover over bars and lines for more information; remove certain categories of lines/bars to focus on an area of interest; and also zoom in on particular parts of the graph.

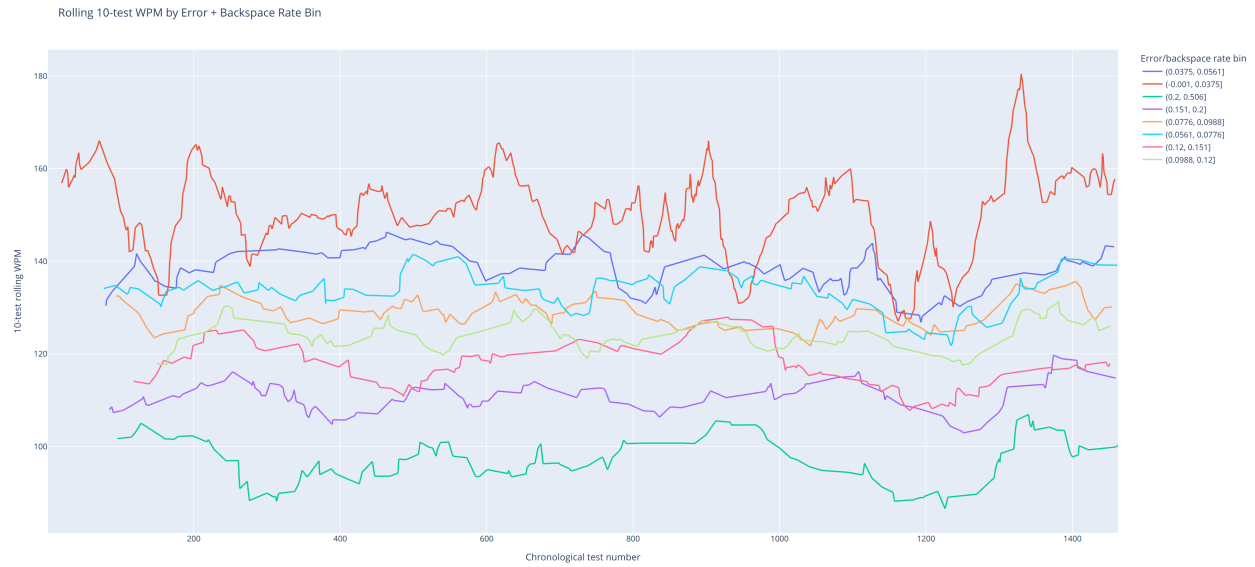
TTTB generates around 29 separate single-player visualizations and 4 multiplayer visualizations. They help you make sense of your average WPM; your accuracy; and your progress in typing through the whole Bible.

Here are some sample PNG screenshots: (Unlike the original HTML copies, these aren't interactive.)

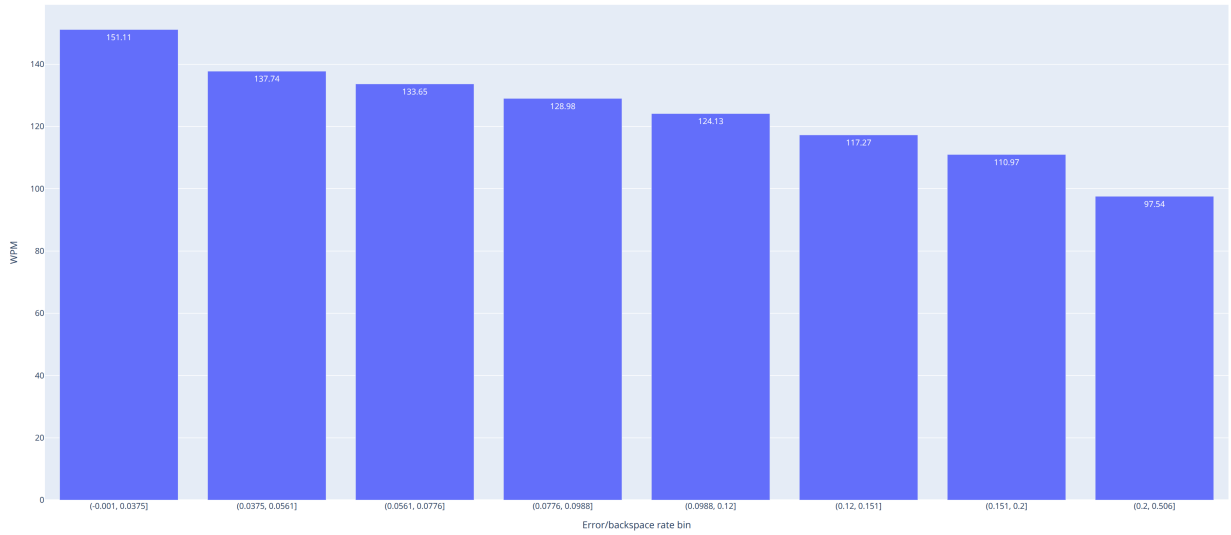




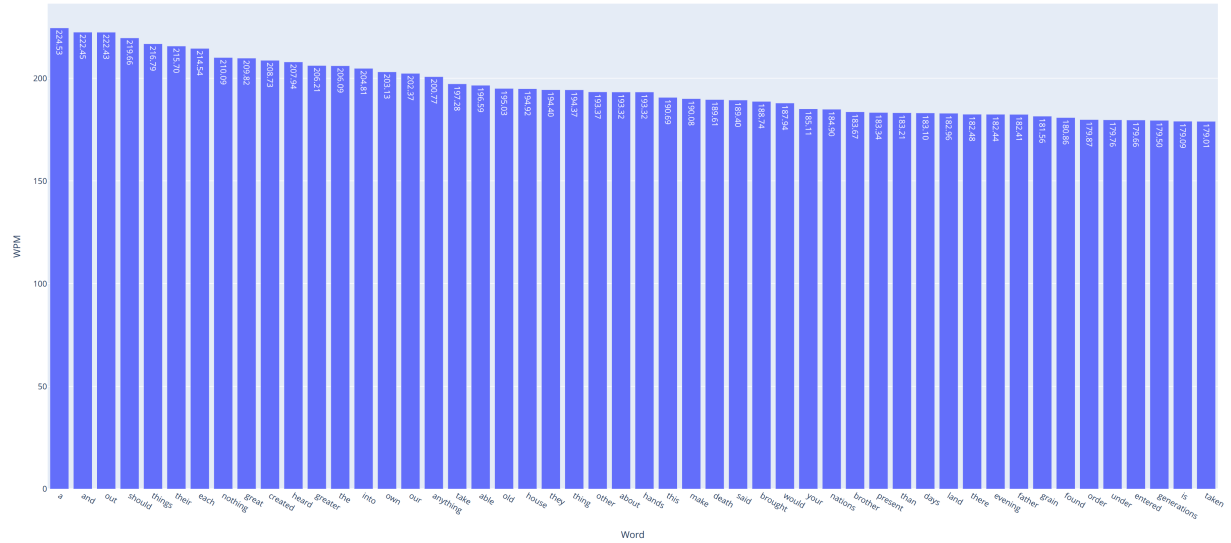

Fastest Tests
WPM
Rank
Error and Backspace Rate
0
50
100
150
200
20
40
60
80
100
0.01
0.02
0.03
0.04
0.05
0.06

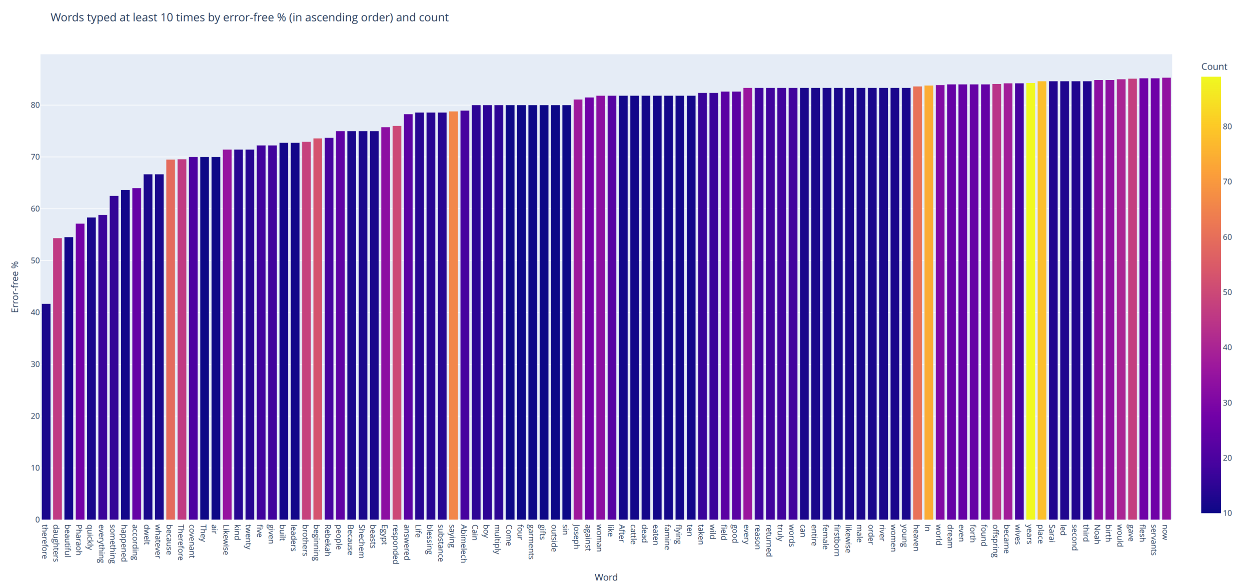

Rolling 10-test WPM by Error + Backspace Rate Bin
10-test rolling WPM
Chronological test number
Error/backspace rate bin
(0.0375, 0.0561]
(-0.001, 0.0375]
(0.2, 0.506]
(0.151, 0.2]
(0.0776, 0.0988]
(0.0561, 0.0776]
(0.12, 0.151]
(0.0988, 0.12]
100
120
140
160
180
200
400
600
800
1000
1200
1400

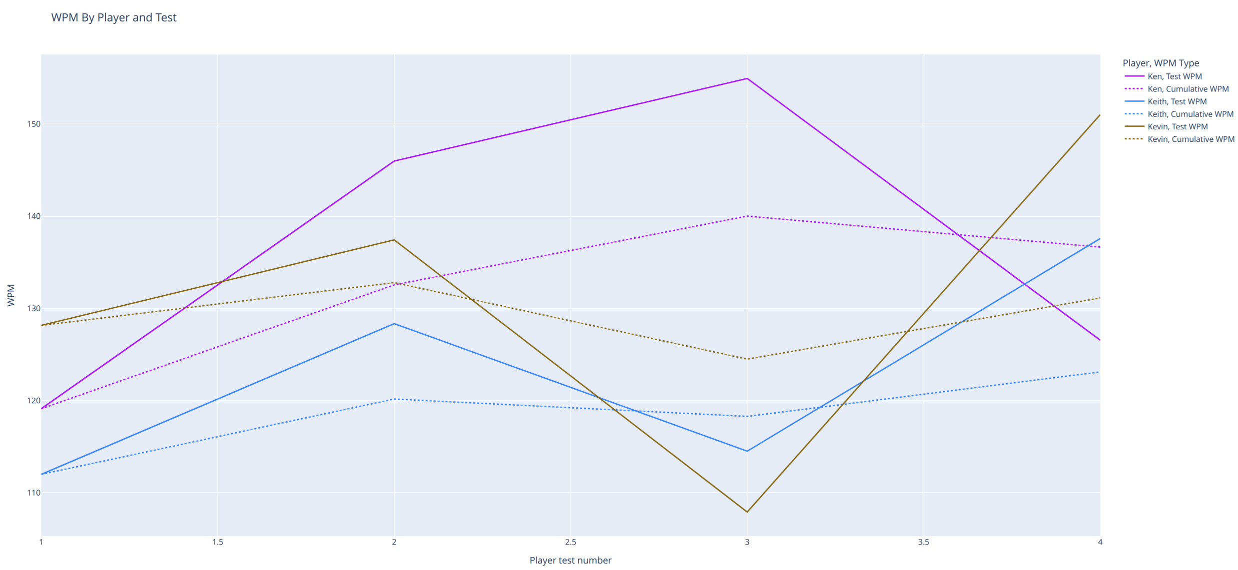
Mean WPM by Accuracy Bin

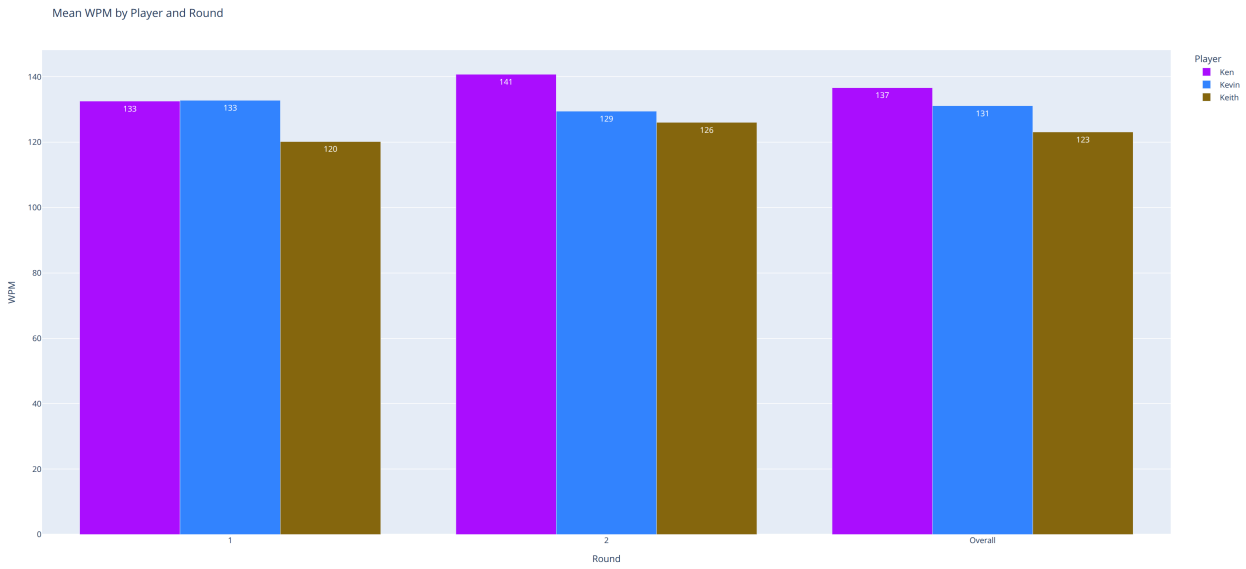


Highest word-level mean WPM for words typed at least 10 times




Words typed at least 10 times by error-free % (in ascending order) and count
Error-free %
0
10
20
30
40
50
60
70
80
Word
therefore
daughters
beautiful
Pharaoh
quickly
everything
something
happened
according
dwelt
whatever
because
Therefore
servant
They
air
Likewise
kind
twenty
five
given
built
leaders
brothers
beginning
Rebekah
people
Because
Shechem
beasts
Egypt
responded
answered
Life
blessing
substance
saying
Abimelech
Cain
boy
multiply
Come
four
garments
gifts
outside
sin
Joseph
against
woman
like
After
cattle
dead
eaten
famine
flying
ten
taken
wild
field
good
every
reason
returned
truly
words
can
entire
female
firstborn
likewise
male
order
river
women
young
heaven
In
world
dream
even
forth
found
offspring
became
wives
years
place
Sarai
led
second
third
Noah
birth
would
gave
flesh
servants
now
Count
10
20
30
40
50
60
70
80


WPM By Player and Test
WPM
110
120
130
140
150
Player test number
1
1.5
2
2.5
3
3.5
4
Player, WPM Type
Ken, Test WPM
Ken, Cumulative WPM
Keith, Test WPM
Keith, Cumulative WPM
Kevin, Test WPM
Kevin, Cumulative WPM



Updating your CPDB_for_TTTB.csv file to incorporate revised copies of verses while still retaining (at least some) of your test results

There are two main reasons why you might need to update the verses within your CPDB_for_TTTB.csv file. First, even months after first releasing TTTB, I'm finding certain issues with my copies of the verses (e.g. trailing whitespaces, HTML tags, etc.) that need to be fixed. (The verses were originally meant for HTML display, so I needed to make a number of revisions so that they would be suitable for a typing test game. It turns out that my initial set of revisions was was sufficient!)

Second, Ronald L. Conte Jr., who translated and edited the Catholic Public Domain Bible, has been making updates to the verses here and there. You may wish to update your own copy of the Bible verses following these edits.

Because I've had to update my own CPDB_for_TTTB.csv file a few times, I created a Jupyter Notebook (cpdb_version_updater.ipynb, available within the CPDB_Version_Updater/ folder) to help simplify this process. Make sure to read the instructions contained at the top of the notebook before running it–and note that there's a decent amount of manual work needed to replace older verses with newer ones. (This is by design, since you wouldn't want to accidentally lose valuable test data!) Also note that the script will *remove* any test and best-WPM data within this file for verses that don't match their previous versions, since I think it's best for that data to always reflect the most recent verse in this file. However, you could modify the notebook to keep those results intact if you so choose.

In order to complete this process, of course, you'll need a newer copy of the CPDB_for_TTTB.csv file. One option is to retrieve it from CPDB_for_TTTB.csv within this GitHub folder; the 'Last commit date' column will let you determine how recently this file has been updated.

Another option would be to use a Python notebook to download the corresponding Bible files from sacredbible.org, then reformat them for use within TTTB. (This is the process that I myself use to update the CPDB_for_TTTB.csv file.) However, I've excluded it from the repository because I didn't want to place unnecessary bandwidth burdens on sacredbible.org.

However, if you've noticed a recent change (i.e. one after 2025-02-26, when the Catholic Public Domain Version was last updated as of 2025-12-02), let me know by reporting an issue or sending a message (see below). I'll then be happy to rerun my corresponding notebook and update the CPDB_for_TTTB.csv file in this GitHub repository's Catholic_Public_Domain_Bible folder.

Reporting issues

I have completed over 1,400 tests on Type Through the Bible so far, along with quite a few multiplayer rounds. However, it's still possible (and perhaps, for a hobby project like this, likely) that bugs, typos, and other issues will appear.

Please make me aware of any issues, large or small, by posting them within https://github.com/kburchfiel/cpp_tttb/issues . If you don't have a GitHub account, you can also email me at dev@kburchfiel.com . (Please include 'TTTB' in your subject so that I know your message is related to this game.)

You are also welcome to use these channels to submit feature requests and improvements; however, since the time I have to work on this project is limited, I may not be able to implement your idea.

Development Notes

I had created an earlier Bible typing game in Python, but given my interest in learning how to apply C++ to create games, I figured that a C++ version of this game would be a great learning opportunity. Although I didn't reference the Python version of this game when writing this code, my experiences with that project certainly influenced the current one.

This project was a lot of fun to work on. While it doesn't feature a GUI, it *was* a good opportunity to for me to get reacquainted with C++–and to gain experience with importing and exporting CSV data. I hope that you enjoy playing it as much as I did coding it!

As with my other GitHub projects, I chose not to use generative AI tools when creating Type Through the Bible. I wanted to learn how to perform these tasks in C++ (or in a C++ library) rather than simply learn how to get an AI tool to create them.

This code also makes extensive use of the following open-source libraries:

1. Vincent La's CSV parser (https://github.com/vincentlaucsb/csv-parser)
2. CPP-Terminal (https://github.com/jupyter-xeus/cpp-terminal)

In addition, this program uses the Catholic Public Domain Version of the Bible that Ronald L. Conte put together. This Bible can be found at https://sacredbible.org/catholic/ I last updated my local copy of this Bible (whose text does get updated periodically) around June 10, 2025.

Acknowledgments

I'd like to give special thanks to Ronald L. Conte Jr. for his work on the Catholic Public Domain Version of the Bible (the translation that TTTB uses).

I am also grateful to Vincent La for his excellent C++ csv parser and to the team behind the cpp-terminal library. These two libraries play a major role in C++ TTTB. I'm also grateful to the Plotly team for their excellent Python visualization library.

Finally, this game is dedicated to my wife, Allie. I am very grateful for her patience and understanding as I worked to put it together! My multiplayer gameplay sessions with her (yes, she was kind enough to play it with me) also helped me refine the code and improve the OSX release.

Blessed Carlo Acutis, pray for us!

Appendix: Compilation Instructions

If you'd like to modify or build upon this game, you'll probably need to compile it first. (Note that there are two separate scripts that you'll need to compile: tttb.cpp, the game's main C++ file, and tttb_py_installer.py, a Python file.)

The following compilation instructions apply to Linux, OSX, and Windows, though the steps will differ slightly across platforms. Please let me know if you encounter any issues.

1. Download CMake if you haven't already.
2. Clone the cpp-terminal and csv-parser libraries, both of which are permissively licensed, and build them using CMake. **Note**: I recommend switching the BUILD_SHARED_LIBRARIES option within the cpp-terminal CMakeLists.txt to OFF for all platforms in order to make your resulting code more portable.

(To build each library using CMake, (a) create a 'build' folder within the folder on your computer that contains the library; (b) navigate within this folder using your terminal; (c) run `cmake ..` in preparation for the build; and then, if that command was successful, (d) run `cmake --build .` Don't forget the space and period at the end!)

3. Copy these folders to a directory of your choice, or leave them in your downloads folder if you prefer.
4. Clone Type Through the Bible and move it to a directory of your choice.
5. **IMPORTANT**: Within TTTB's CMakeLists.txt file, you *must* replace the existing paths to my copies of the cpp-terminal and csv-parser libraries within the add_subdirectory() calls with paths to *your* copies of these folders (at least for the platform(s) for which you're building TTTB). Within each add_subdirectory() call, the first path is to the actual 3rd-party-library's location on your computer; the second path tells CMake where to place some additional files related to that library. (You don't need to create this second folder on your system; CMake will create it automatically.)
6. Navigate into TTTB's build/ folder.
7. Use CMake to build TTTB. (See above steps for reference if needed.) Once it has been build, copy the resulting executable (e.g. `tttb` or `tttb.exe` on Windows) into your build folder. (Windows will likely place it into a separate subfolder, but it must be placed within your main build folder in order for the relative paths used by the program to work correctly.)
8. Next, you'll need to build an executable version of the tttb_py_complement.py file; this can be accomplished via Pyinstaller. First, you'll want to download Pyinstaller if you don't have it already. Also, download Python (i.e. via Miniforge if it's not already on your system.
9. Make sure that **Pandas**, **Numpy**, and **Plotly** are all installed within the Python environment that you plan to use to run Pyinstaller. It's helpful, but not necessary (to the best of my knowledge), to have them within your base environment. (These can be downloaded via conda-forge.) Otherwise, the executable version of your tttb_py_complement.py file won't work correctly.

Note: You may also want to install JupyterLab (e.g. via `conda install jupyterlab`) so that you can modify, if needed, the .ipynb notebooks contained within the source code. (Just make sure to export these updated notebooks to a .py file so that they can get processed correctly by create_release_folder.py and your pyinstaller call.)

10. Once you have Python and Pyinstaller set up, first activate your environment of choice within your terminal (e.g. by running `conda activate tttb_env`; replace 'tttb_env' with whatever your environment name happens to be.) Next, navigate to TTTB's 'build' folder and run `pyinstaller tttb_py_complement.py`. This will create an executable version of this file, along with an '_internal' folder that contains important library components, into a 'dist/tttb_py_complement subfolder' within your 'build' folder. **Cut and paste both the _internal folder and the tttb_py_complement executable into your build folder, as that's where the program will look for them.**

Note: If you encounter issues with Pyinstaller, consider (1) creating a brand new conda environment that contains *only* the latest copies of the libraries required for the code to run (i.e. Pyinstaller, Numpy, Pandas, and Plotly) and their dependents, then (2) rerunning your Pyinstaller command from within that conda environment. (I've found, based on my tests, that Pyinstaller will use the versions of the libraries present within the conda environment from which you activate it. Therefore, if your base environment is giving you trouble, simply create a new environment instead.)

Here's a sample set of terminal commands that can help you accomplish these steps:

[Conda will need to be added to your path in order for the following step to work; this was probably already completed when you installed Miniforge.]

a. `conda activate base`

b. `conda create -n tttb_analysis python pandas numpy plotly pyinstaller`

[After running the above code, make sure to enter 'y' after Conda asks whether you wish to download and install these packages (along with their dependencies.]

c. `conda activate tttb_analysis`

[Navigating to the location of the Git clone's build folder: (Replace this path with the path to your own Type Through the Bible build folder.].

d. `cd '/home/kjb3/D1V1/Documents/!Dell64docs/Programming/CPP/cpp_tttb/build'`

[Running pyinstaller:]

e. `pyinstaller tttb_py_complement.py`

10. Navigate up to the main cpp_tttb folder (e.g. via `cd ..` in your terminal, assuming you're still in the build folder) and run `python create_release_folder.py neither`. This will create a new copy of TTTB with blank output files rather than the existing files within the TTTB directory.

Note: The create_release_folder.py file also contains code that can help automate the process of building and (where necessary) moving your C++ and Python executables. (For instance, you can pass `both` as a corresponding argument to the file rather than `neither`, the default, in order to compile the executables. See the source code for more documentation and details.) However, I strongly recommend performing these steps outside of this file the first time around so that you can more easily identify and debug any issues. Also note that, if you set 'both' or 'py' as your argument, you'll want to call this script within the Python environment that you want to use for Pyinstaller.)

If you do want to try compiling both the C++ and Python executables via the 'create_release_folder.py' script, first complete steps a, b, and c in the above list. Next:

d. Navigate to TTTB's root folder via the following command: (Make sure to replace the path shown with the path to your own copy of TTTB. Note that the root folder won't necessarily be named 'cpp_tttb.')

```
cd '/home/kjb3/D1V1/Documents/!Dell64docs/Programming/CPP/cpp_tttb'
```

e. Execute the following command:

```
python create_release_folder.py both
```

11. Try launching the TTTB executable–if it works, congratulations! If not, double-check the steps and/or reach out to me with any questions.